2021 International Conference on Unmanned Aircraft Systems (ICUAS)
Athens, Greece. June 15-18, 2021

Which Is the Best Real-Time Operating System for Drones? Evaluation of the Real-Time Characteristics of NuttX and ChibiOS

Mingyang Zhang[1] Martin Timmerman[2] Luc Perneel[2] Toon Goedemé[1]

***Abstract*— With the evolvement of various drone platforms, the constraints of such embedded systems are getting more and more strict and specific. As the low-level technology, correct behaviours and decent performance of an RTOS play a vital role in guaranteeing such constraints. This paper studies the real-time performance and behaviour of two popular RTOSs used in drones, presenting both quantitative and qualitative results. We also exploited the source code to locate the origin of possible misbehaviours. By analysing the results, we draw several conclusions on the comparison of NuttX and ChibiOS, which can be useful for drone developers and users. NuttX has a vital defect in the implementation of priority inheritance. And in most tests, ChibiOS beats NuttX, therefore, it is the winner in this comparison.**

I. Introduction

Ardupilot and PX4 are the two most used flight controller firmwares nowadays and they were both developed based on the NuttX Real-Time Operating System (RTOS). However, Ardupilot switched to ChibiOS since 2018 while PX4 still sticks to NuttX. A reliable flight controller firmware should meet strict real-time constraints, especially having a predictable behaviour. Therefore, the real-time characteristics of the operating system are critical. But, up to our knowledge, there is no such existing research benchmarking comparing behaviours and performance of these two RTOSs.

In this paper, we present our evaluation results which are yielded by adopting a benchmark framework composed of 4 test suites developed by the Dedicated Systems Experts company: RTOS system tick tests, thread tests, mutex tests and semaphore tests. These test suites introduce both qualitative and quantitative indicators to understand the advantages and drawbacks of each RTOS thoroughly. All tests are done on the same hardware platform to get comparable results.

This paper is organised as follows. Section 2 gives an overview of the tested RTOSs and the versions used for testing. Section 3 explains the hardware and software architecture used in this benchmark. Section 4 illustrates the results and comparisons between the two RTOSs. In section 5 we will analyse the results, draw the conclusions and list some future work.

[1]Mingyang Zhang and Toon Goedemé are with PSI-EAVISE research group, Department of Electrical Engineering, KU Leuven, Leuven, Belgium. {mingyang.zhang, toon.goedeme}@kuleuven.be

[2]Martin Timmerman and Luc Perneel are with Dedicated Systems Experts NV, Beersel, Belgium. m.timmerman@dedicated-systems.net l.perneel@luperco.com

This work is supported by the Chinese Scholarship Council (CSC) and the FWO (SBO Project Omnidrone).

II. Related work

A. RTOS Test and Comparison

There is not a universal testing terminology for RTOSs. Kumar Reddy et al. presented their work [1] comparing the scheduling latency of FreeRTOS and ChibiOS in 2014. Lee et al. [2] adopted the benchmark developed by EEMBC [3] and designed different threads with different focuses to test MRTOS and VxWorks. Some threads are floating-point computation-intensive, and some are mutex usage intensive etc. However, the benchmark of EEMBC is more used to benchmarking microprocessors instead of RTOSs.

As for the testing of operating systems used for drones, there is only limited previous work. André de Matos Pedro et al. [4] published their research on runtime verification of PX4 and Ardupilot in 2018 at which time the two autopilot systems were still both based on NuttX.

None of the work mentioned performed a thorough test on the essential real-time characteristics of NuttX and ChibiOS, the two most used RTOSs in drones. In this paper, we present the test methodology and the testing results of these two RTOSs. Such comparison can help RTOS and autopilot system developers improve their projects and more importantly, help drone designers to pick better RTOS and design better software on drones.

B. Relative RTOSs Overview

NuttX was developed by Gregory Nutt with a focus on POSIX compliance and low computational weight. It is written exclusively in C language and thus supports MCUs varying from antiquated 8-bit MCUs to high-speed 32-bit ARM cores, or even 64-bit architectures [5].

There are several characteristics of NuttX making it an attractive choice among its competitors. It was previously under the BSD license and currently the Apache License 2.0. Hence it is friendly to commercial use. NuttX is currently undergoing incubation at The Apache Software Foundation and it is widely deployed on embedded devices, especially drones. All these make NuttX an RTOS with continuous and robust support from both developers and the community. Most importantly, NuttX perfectly supports FIFO and round-robin pre-emptive scheduling policies. The fact to have a pre-emptive scheduling mechanism is a minimum requirement for an operating system to meet deadlines, permitting for the application of rate monotonic scheduling principle and thus making a hard real-time approach possible [6]. In this paper, we used NuttX version 9.0 for the tests.

2021 International Conference on Unmanned Aircraft Systems (ICUAS) | 978-1-6654-1535-4/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICUAS51884.2021.9476878

Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 09,2022 at 16:31:16 UTC from IEEE Xplore. Restrictions apply.

ChibiOS was developed by Giovanni Di Sirio in 2007. It is released under the GPL3/Apache2.0 license for open source projects but is also available under licenses for commercial use. ChibiOS is a complete development environment for embedded applications including an RTOS, a Hardware Abstraction Layer (HAL), peripheral drivers, support files and a development environment [7]. Specifically, the ARM development environment ChibiStudio makes it easy for developers to modify, compile and debug source code.

For an RTOS to be deployed on constrained devices, to be lightweight is a key feature. The kernel size of ChibiOS(V20) is only 5KB compared with 20KB of NuttX(V9) [8], [5]. The features of ChibiOS can be easily tailored to fit embedded scenarios. It is worth mentioning that the HAL comes with ChibiOS provides high-level API to peripherals. It makes the embedded applications more portable. We used ChibiOS RT 6.1.0 for the testing.

III. Experimental Setup

In this section, we will describe the architecture we used for the benchmarking system and explain how the experiments are done.

A. System Topology

Fig. 1 demonstrates the testing system architecture used for testing. In the tests, we used a STM32F429I-DISC1 evaluation board with a STM32F429ZIT6 MCU. Some important features of this board are:

- 32-bit STM32F429ZIT6 MCU with a 180MHz Arm Cortex-M4 core (CPU does not have cache)
- Embedded STLINK/V2-B debug tool with virtual COM port capability
- On-Chip Memory: 2 Mbytes of Flash memory organised into two banks allowing read-while-write and 256 Kbytes of RAM including 64-KB of CCM (core coupled memory) data RAM
- An external 64-Mbit SDRAM
- USB OTG with Micro-AB connector
- Two 16-bit advanced-control timers, eight general-purpose timers and an RTOS dedicated system tick timer with a 24-bit down counter

Since the MCU STM32F429 and other similar MCUs in its family are widely used in embedded designs, especially drone flight controllers, drivers for this MCU are readily available for both RTOSs.

The RTOS evaluation application runs above the RTOS layer. Compilations in this project for both RTOSs are done by GNU Arm Embedded Toolchain Version 9 for Linux. The debug tool for NuttX was GDB plus OpenOCD in Linux. For ChibiOS, we used ChibiStudio because it has already GDB and OpenOCD in it and its GUI makes it easier to manipulate breakpoints during debugging. The flashing tool was st-flash in the stlink-tools package in Ubuntu Linux.

USB and UART connections are needed during each test, but the communication between PC and the STM32 board can influence the test process thus makes the time trace of test erroneous. Therefore, we avoided the occurrence of any USB or UART communication during the test phases.

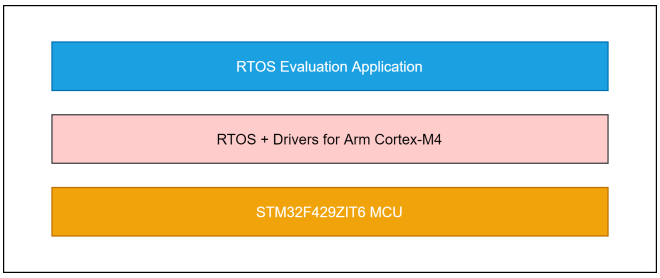

Fig. 1. Testing architecture for NuttX and ChibiOS

B. Evaluation framework

The testing framework used in this paper was originally developed by Luc Perneel, Martin Timmerman et al. to test real-time properties of RTOSs [9], [10], [11].

The framework describes the real-time tests needed to be done for an RTOS. Each test has its own specific algorithm to run a task focusing on a particular aspect of the tested RTOS. Based on the results of different tests, one can draw an overall conclusion on the tested RTOS to decide if it is suitable for a specific real-time project.

We adapted the testing framework in this paper, dependent on the testing hardware platform and each tested RTOS.

C. Test Implementation Overview

A hard real-time system must meet its deadlines [6]. To achieve this, a good RTOS should assure predictable behaviour and have low latencies. The testing methodology provides for validating both aspects on the two RTOSs. First, we used some tests to verify the behaviours. Without correct behaviours, an RTOS cannot guarantee meeting deadlines. Then a number of performance tests are conducted to have a better view of the latency/throughput of an RTOS especially verifying these are bound. Generally, the performance shows the capacity or limit of an RTOS. In other words, we measure under what kind of load the RTOS can guarantee meeting deadlines and how short these deadlines may be. With the above principles, we used test patterns which correspond to the mechanisms used in real-time applications in various industries.

The tests are based on measuring a time interval between 2 events occurring in the RTOS. On this platform, the Data Watchpoint and Trace Unit (DWT) integrated in the Arm Cortex-M4 core is used to measure time intervals. On our platform, the timer used runs at 168MHz. Thus, the resolution of time interval measurement is 5.9ns. Therefore, the results in this paper are rounded up to 10ns.

Fig. 2 illustrates the process of a typical performance test. The test first creates a thread with high priority. Once created, the thread will conduct some preparation work. Then it repeats the following loop: write down start time, perform a test action and write down end time. In Fig. 2 t_1 and t_3 represent the starting time and ending time of one test loop,

Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 09,2022 at 16:31:16 UTC from IEEE Xplore. Restrictions apply.

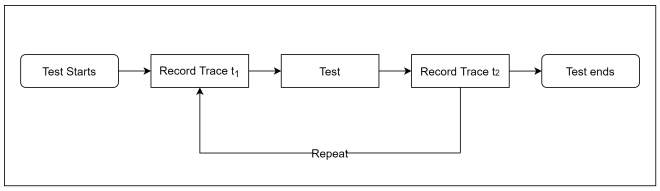

Fig. 2. A performance test process

such that $(t_3 - t_1)$ is the tested time interval. This execution is repeated for several thousand times to make a statistical analysis possible and especially to look for a maximum value of the time interval and at the mean time to verify that this maximum is bound. Of course one should loop for an infinite amount of time to be sure to detect the maximum bounded value. But for practical reasons, such as buffer space and available test time, we limit the number of loops.

IV. Test detail description

The testing framework we adopted consists of 4 test scenarios, described in detail in the subsections below. First, the overheads caused by both tracing and system tick are measured. Then the correctness of thread behaviour is checked, followed by the temporal tests focused on the performance of threads. Similarly, both behavioural tests and performance tests are done for mutexes and semaphores.

A. Overhead Tests

In this part, we measure two kinds of possible overheads. The first one is the overhead caused by tracing operations. This kind of overhead needs to be deducted when performing time interval measurements. Another kind of overhead is caused by system ticks. This is due to the fact that the ticker interrupt is put on the highest interrupt level in most hardware. If the hardware used in a design has this characteristic, then this overhead will always be there and impact real-time behaviour. One may avoid this by putting the ticker interrupt at lower interrupt levels, but then the ticker should not be used for time related system calls in the application. In this paper, we will exclude this ticker overhead from the measured data considering the designer will take care of lowering the interrupt level of the ticker routine.

1) Tracing Overhead: As shown in Fig. 2, the test starts after t_1 is written down and ends after t_3 is written down. But in the real implementation, there will be some delays after the "start" and before the "end". These delays are mainly introduced by the memory writings of t_1 and t_3. In this paper, we call these delays "post-start" and "pre-end" delays correspondingly.

The tracing overhead test aims to test the total delay summing up the post-start delay and pre-end delay to guarantee the accuracy of all other tests. The test process is similar to Fig. 2; the only difference is that the "Test" execution part does not exist in the tracing overhead test. In this test, t_3 is recorded right after t_1 is recorded. In this way, the difference between t_3 and t_1 will be the overall delay which is the tracing overhead. It is also checked that this tracing overhead remains low in order to accept that this is not influencing (too much) the real behaviour of the system.

2) System Tick Frequency and Overhead: In a classic OS, the operating system wakes up and performs the scheduling periodically. Such a system timer interrupt is called a system tick. In an event driven application, this is not needed. However most (RT)OS are using the ticker for interval timings and as such cannot always be avoided. In that case, the use of this system ticks brings overhead and increases power consumption. It is also worth pointing out that both tested RTOSs support a tickless mode in which a dynamic interrupt will be applied instead of the periodic one.

In this test, we adopt a periodic system tick interrupt in both RTOSs to measure the overhead introduced by these system ticks.

Usually on an embedded hardware platform, the clock interrupt task runs at the highest interrupt priority. Since it can interrupt any other task, the system tick routine will happen during all the tests in this paper, introducing an overhead. In both tests RTOSs, system tick is used to specify at which frequency the scheduler reschedules the tasks. For all the tests of this paper, the system tick frequency for both RTOSs is set at 100Hz.

Fig. 3 shows the process of system tick overhead testing. Each test loop does some simple calculation which takes certain CPU time. This task can be only interrupted by a system tick. When a system tick happens, a context switch storing the state of the current task will consume some CPU time. In the timer tick interrupt handler, if waking up a thread is needed, the scheduler task will run and reschedule the tasks if needed. Another context switch after the scheduler finishes its task. The overall latency is the overhead of a system tick. In our case, the rescheduling is not needed, so the overhead is minimal.

B. Thread Test Scenarios

In this subsection, we will describe the 4 thread tests. First, we test the correct behaviour of thread creation, then the round-robin scheduling mechanism followed by the thread switch latency, finally the time cost of thread creation and deletion.

1) Thread Creation Behaviour: In a well-designed real-time system, threads should not be created and destroyed dynamically. However, if it happens, a wrong behaviour of a created thread may lead to missed deadlines. We test the following behaviours in this test:

- If the created thread has a lower priority than the creating thread, is it deactivated until the creating thread finishes?

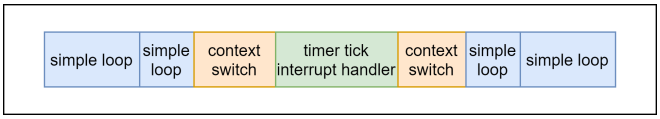

Fig. 3. Time flow of system tick processing

Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 09,2022 at 16:31:16 UTC from IEEE Xplore. Restrictions apply.

- If the created thread has the same priority as the creating thread, is it placed at the head of the ready queue or the tail?
- If the creating thread yields after creating a thread with the same priority, does the created thread run?
- If the created thread has a higher priority than the creating thread, is it activated immediately?

2) Thread Round-robin Behaviour: Assigning the same priority level to a lot of threads while designing a real-time system is a bad practice. However, not all real-time designers can avoid this in designs because it might be not possible to assign a different priority level to all individual threads. Both tested RTOSs have a round-robin implementation for scheduling threads with the same priority.

A scheduler with round-robin policy runs a job for a time slice (sometimes called a scheduling quantum) and then switches to the next job in the run queue [12]. In this behavioural test, every created thread has the same priority level. The created thread enters an infinite loop until the scheduler makes it wait and schedules the next thread with the same priority to run.

3) Thread Switch Latency: This test measures the switch latency between threads at the same priority. The context switch time is mainly affected by two elements. The first factor is the hardware. This factor, however, indicates not only the MCU used in a real-time project but also the operating system code which is specifically designed for the hardware. This depends on the definition of the context by the OS designer. The second part is the time the OS spends in the task scheduler to decide which task to run next. This part of the latency is independent of the hardware and it is the part we focus on in this paper.

In order to reach the limit of the tested RTOSs, different numbers of threads with the same priority are implemented in the test: 2, 4, 8 and 16 threads. The reasons for not increasing the number of threads beyond 16 are that the RAM of this platform is limited and that 16 threads are usually sufficient in a drone flight control system.

To test the switch latency, we want the running thread to yield its resources voluntarily without having a round-robin scheduling interrupt.

The worst-case switch latency is always caused by a system tick in this test. These data are not the focus in this test, so we compare the results of two RTOSs only after excluding the data acquired during a system tick.

4) Thread Creation and Deletion Cost: In this test, we measure the time cost of thread creation and deletion under three different circumstances:

- The created thread never runs. It has a lower priority than the creating thread and is deleted before it gets any chance to run. In a correct implementation, no thread switch should happen in this case.
- The created thread runs and terminates immediately. It has a higher priority than the creating thread. After it starts running, it terminates itself.
- The created thread runs and blocks itself. It also has a higher priority than the creating thread. But instead of terminating itself, it lowers the priority of itself thus gets blocked.

C. Mutex Test Scenario

The mutex test scenario consist of three tests First, a behavioural test to verify if a priority inversion avoidance implementation is available, followed by the performance test of mutex acquisition and release operations. The last test is the mutex creation and deletion time cost.

1) Priority Inversion Avoidance: Priority inversion is a common problem that most operating systems must deal with. Failing on avoiding priority inversion may lead to system failure, especially for a real-time system. One famous example is that the Mars Pathfinder lander kept resetting when performing missions on Mars. The problem was caused by priority inversion, found later by the development team of the lander [13]. Although good application design might avoid priority inversion, designers are only human and might make mistakes. It is therefore better to foresee for a priority inversion avoidance mechanism.

A simple example of how priority inversion happens can be described as following. Suppose T_1, T_2, T_3 are three threads in descending order of priority where T_1 has the highest priority. T_3 acquires a mutex to gain access to some shared resource. T_1 starts to run and pre-empts T_3, then T_1 tries to gain access to the same shared resource being used by T_3, so T_1 gets blocked. T_3 resumes to run but it may get preempted by T_2. In this way, T_2 has a lower priority than T_1, but it may finish before T_1 [14].

There are several solutions to avoid priority inversion. Two of the most used methods in RTOS are priority ceiling and priority inheritance. The latter mechanism is used in both NuttX and ChibiOS. With priority inheritance policy, when a thread at high priority waits for a mutex being owned by another thread, the priority of the mutex owner thread will be boosted to the same level as the high priority thread in the duration of owning the mutex. With the boosted priority, the (originally) low priority thread will not be pre-empted by other medium priority threads anymore.

The test tries to reproduce the above priority inversion scenario in the tested RTOSs. The correct behaviour of the three threads is shown in Fig. 4. To control the timeline of thread execution in this test, two binary semaphores are introduced used by T_2 and T_3, called S_1 and S_2 correspondingly. T_3 runs first and creates the two semaphores with their counters equal to zero. Then T_3 creates a mutex called M_1 and two threads T_1, T_2. T_1 starts running immediately and pre-empts T_3. T_1 tries to acquire semaphore S_1 and gets blocked. Then T_2 runs because it has higher priority than T_3. T_2 tries to acquire the semaphore S_2 and gets blocked. Then T_3 continues running, acquires the mutex and releases semaphore S_1. After that, T_1 tries to acquire the mutex M_1 and gets blocked. T_3 resumes and releases the semaphore S_2. At this time point, if there is a priority inversion avoidance mechanism implemented, then T_2 should not run after the semaphore released because the priority of T_3 has been boosted to the same level as T_1's. After releasing semaphore

Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 09,2022 at 16:31:16 UTC from IEEE Xplore. Restrictions apply.

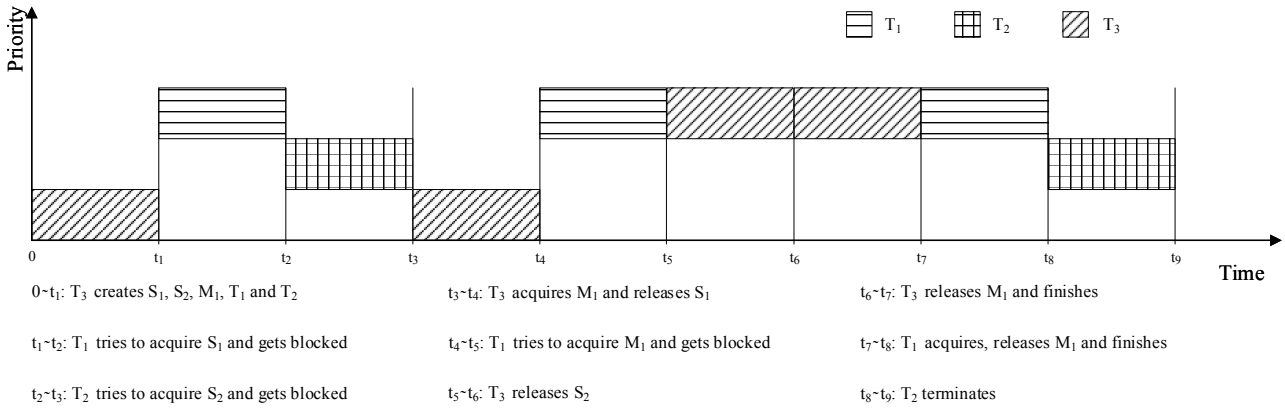

Fig. 4. Priority inversion avoidance test process

S_2, T_3 should continue running and releases the mutex M_1. T_3 finishes its job and terminates itself after releasing M_1. Then T_1 should run, acquires M_1, releases M_1 and finally terminates. T_2 should run and terminates after T_1 terminates.

To check whether an RTOS can pass this test, the states of currently running thread are recorded in a global array. If the order of states is the same as the expected one, then the priority inversion avoidance mechanism is working properly.

2) Mutex Acquisition and Release Performance: In well-designed application software, a thread is asked to acquire a mutex when it accesses critical section. The acquisition and release costs are important parameters for a real-time application.

In this test, the mutex acquisition and release time is measured in two cases: contention- and non-contention case. For the contention case, the test process is the same as the one for priority inversion test but in a loop manner. That means in Fig. 4, the executions between t_1 and t_8 loop many times to measure the average and maximum time costs of mutex acquisition and release.

The mutex acquisition time in contention is the time from the moment the high priority thread tries to acquire the mutex, to the low priority thread getting activated again after the high priority thread gets blocked. Therefore, this time duration includes a thread switch from the high priority thread to the low priority thread. In Fig. 4, this duration is represented by $t_5 \sim t_6$.

The mutex release time in contention is the time from the (originally) low priority thread releases the mutex to the high successfully acquires the mutex. Thus, this time duration also includes a thread switch. In Fig. 4, this time cost is represented by $t_6 \sim t_7$. Note that in this test T_3 does not terminate after releasing the mutex but keeps looping after T_1 gets locked again on the semaphore S_1 again.

The non-contention case of this test is implemented by using only one thread. This thread creates a mutex, acquires it, releases it, and repeat the process many times. This test is equally as important as the in-contention mutex performance test because most of the time, a real-time application uses non-contended locks. Only in rare cases the mutex will be taken by another thread. Therefore, a well-designed RTOS must also assure that the mutex acquires and releases in a short, bounded time in non-contention case.

3) Mutex Creation and Destruction Performance: This test measures the time it takes to create and destroy a mutex.

Most of the time, dynamic allocation of system objects is not done in real-time systems. However, a notable exception can be a mutex. In cases where fine-grained locking is used (which is useful to avoid contention in most cases), locks could be dynamically added and released in the system. Therefore, this test case is more important than the other creation/destruction test cases.

Remark that in case the mutex cannot be created in place, all required mutexes are pre-allocated upfront (at system startup) just like any other dynamic object system in a system with real-time requirements. Therefore, this test shows if a mutex can be initialised (created) in place or not.

The destruction overhead is tested in two cases. In the used mutex case, the mutex is used before it is deleted. And in the never used case, the mutex is deleted right after it is created.

D. Semaphore Test Scenario

Similarly, three tests for semaphores are available. Firstly, it is tested if a semaphore locks and releases correctly. Then the time cost of semaphore acquisition and release is measured. The last test in this part is the semaphore creation and deletion overhead test.

1) Semaphore Locking and Releasing Behaviour Test: In this test, we check whether the counting semaphores in the tested RTOSs lock and release as expected.

In the common definition of a counting semaphore, the signalling V() call increases its counter, and the waiting P() call decreases its counter. When the counter is zero, the P() call will get the calling thread blocked. And when the counter becomes greater than zero, the scheduler in the OS should reschedule the thread blocked on the semaphore with the highest priority to run. Therefore, to test the semaphore locking behaviour, the test checks the following aspects:

- A thread trying to call P() on a semaphore only gets blocked when the semaphore counter is zero;

Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 09,2022 at 16:31:16 UTC from IEEE Xplore. Restrictions apply.

- The V() call on a semaphore increases the counter of that semaphore;
- When the counter of a semaphore is zero, the scheduler of the OS should arrange a rescheduling to unblock the threads waiting for the semaphore if a P() operation is called to that semaphore.

The above test algorithm uses two threads T_1 and T_2 where T_1 has a higher priority than T_2. The test process is shown in Fig. 5. T_1 runs first and creates a semaphore with its counter equals to 0, and a thread T_2 which is blocked because of its lower priority. Then T_1 tries to decrease the semaphore counter and gets blocked. T_2 should then get activated and increase the semaphore counter. After that, T_1 should be scheduled to run again and then increase the semaphore counter three times. Then it decreases the counter three times which should neither block itself nor activate T_2. T_1 then tries to decrease the semaphore counter again and gets blocked. T_2 then increases the semaphore again, which activates T_1. After that, the two threads will terminate themselves to end the test.

If the two threads behave exactly as described above and shown in Fig. 5, then the system under test succeeds this test. Otherwise, there is some bug in the tested RTOS.

In the releasing behaviour test, we test only one thing: whether the highest priority thread among the threads blocked on a semaphore is released if a signal operation is called on that semaphore. The unblocking of the highest priority thread should not be affected by the chronological order of the wait operations. Thus, the test is done with the highest priority thread waiting on the semaphore first and with the lowest priority thread waiting on the semaphore first, separately.

To realise the above test, an application using 3 threads T_1, T_2 and T_3 with ascending order of priority are used. In the first scenario, T_1 activates T_2 first and T_2 tries to acquire the semaphore before T_3. In the second scenario, the thread activation and semaphore acquisition operations are in the opposite order. After both T_2 and T_3 are blocked on the semaphore, T_1 signals the semaphore. T_3 should then be activated regardless of the order of semaphore acquisition operations of T_2 and T_3.

2) Semaphore Acquisition and Release Performance: Like the tests for mutexes, the acquisition and release performance of semaphores is checked in two cases: with and without contention.

For the non-contention scenario, simply one thread is used to perform the test. The thread creates a semaphore with the initial counter equal to 1. Then the thread enters a loop where it acquires and releases the semaphore while recording the time cost of the operations. The loop will be executed many times to get the average and maximum time cost of semaphore acquisition and release operations.

To test the semaphore acquisition and release performance when contended, a test involving 50 threads is used. T_0 is the creating thread with the lowest priority while T_1 to T_{49} have ascending priorities. T_0 firstly creates the semaphore with its initial counter equal to 0. Then T_{49} to T_1 is created one by one. Each of the created threads tries to acquire the semaphore after being created thus gets locked. While creating a thread, the time from the created thread trying to acquire the semaphore to the activation of T_0 is measured as the semaphore acquisition time cost. As in the mutex tests, this time duration also includes a thread switch. After all the threads are created and blocked on the semaphore, T_0 signals the semaphore 49 times. After each release operation, the thread with the highest priority in the waiting list should be activated. The time from T_0 calls V() on the semaphore to the highest priority blocked thread getting activated is measured as the release operation overhead.

3) Semaphore Creation and Destruction Overhead: In this test, we measure the time it takes to create and destruct a semaphore. Typically, semaphores should be created once at start-up and not dynamically at run time, but the application designer might decide otherwise

This test uses only one thread. The thread creates a semaphore and records the time used on semaphore creation. Then, in the scenario of testing semaphore deletion time, it tries to acquire the semaphore then releases it. In this normally never-used scenario, the thread does nothing before deleting the semaphore. In the end, the thread deletes the semaphore and writes down the time consumed on the destruction operation.

V. Testing results and analysis

As the goal of this paper is to benchmark NuttX and ChibiOS, we used the above described testing framework on these two RTOSs. Below, the achieved test results are detailed. As described earlier, these experiments were done on a STM32F429I-DISC1 evaluation board with a STM32F429ZIT6 MCU. We will organise these results in calibration tests, behavioural tests and performance tests.

A. Calibration Tests Results

The tracing overheads of the two tested RTOSs are shown in Table I. We can see that ChibiOS has shorter tracing overhead than NuttX. This overhead will be deducted from all the results of performance tests.

B. Behavioural Tests Results

All the results of behavioural tests, concerning threads, mutexes and semaphores are shown in Table II.

As we can see, NuttX succeeds in all thread creation behaviour tests. For the second test, instead of putting the created thread at the tail of the ready queue like it in ChibiOS, the scheduler of NuttX places the new thread at the head of the ready queue. In other words, the created thread starts to run immediately after being created if it has the same priority as the creating thread. Such policy is not erroneous, but should be known by the application designer. ChibiOS succeeds in all the thread creation behaviour tests.

Both NuttX and ChibiOS succeed in the round-robin behavioural test. The time slice used in this test for both RTOSs is 200ms. It is important to mention that the time slice of round-robin scheduling can be directly set in NuttX

Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 09,2022 at 16:31:16 UTC from IEEE Xplore. Restrictions apply.

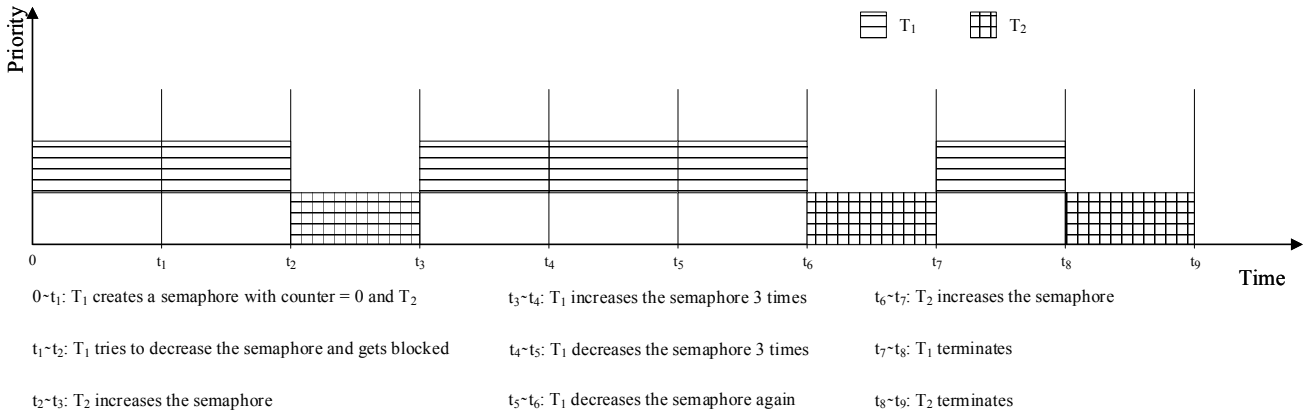

Fig. 5. Semaphore waiting test

TABLE I
CALIBRATION TEST RESULTS

Calibration Tests	NuttX			ChibiOS		
	Min	Avg	Max	Min	Avg	Max
Tracing overhead (timer ticks)	31	31	37	**20**	**20**	**22**

in microseconds, while in ChibiOS you must set it in system ticks which makes it inconvenient especially when changing of the system tick cycle is needed.

As for the priority inversion avoidance test, both NuttX and ChibiOS passed. However, with NuttX, we found that there is no priority inheritance implemented for threads blocked on a mutex but only on a binary semaphore. By replacing the mutex in the test with a binary semaphore (with the initial counter = 1), the priority inheritance works perfectly. This problem is not explained in any official documents of NuttX but should be known by the application designer. It is probably because mutexes were introduced later to the POSIX RT extensions and NuttX did not change its implementation of priority inversion avoidance algorithm to a mutex-based one. This misuse of binary semaphores on priority inheritance may cause confusions for NuttX users and possibly lead to wrong designs.

For the semaphore locking and releasing behaviour: both NuttX and ChibiOS passed the behaviour test without any error.

C. Performance Tests Results

All the results of performance tests are in Table III. The time cost of a system tick interrupt routine is firstly listed. Then all the measured timings of the tread, mutex and semaphore performance tests are shown. The best values of each measurement are again indicated with bold numbers.

We can see that the system tick overhead of ChibiOS is larger than that of NuttX. But the overheads of both RTOSs are acceptable considering the MCU clocking frequency and common real-time requirements. What is more, the variance of system tick overhead for both RTOSs is minimal, which is not only good for an RTOS to be stable, but the maximum values are close to the average values and are most probably be bound.

In the test of switch latency of ChibiOS, we need to change the OS settings before compiling the firmware to deactivate round-robin scheduling. In other words, it cannot be changed when the RTOS is running. However, as NuttX is POSIX compliant, it allows designers to specify the scheduling policy as either “SCHED_FIFO” or “SCHED_RR”, FIFO policy and round-robin policy respectively, by calling the function “pthread_attr_setschedpolicy” when creating the thread. NuttX is more flexible than ChibiOS if a real-time designer wants to assign different scheduling policies to tasks within one system. The results show that the thread switch latency of NuttX is much higher than the one of ChibiOS on this specific platform. There are several reasons why it is the case. First, ChibiOS is a lightweight real-time operating system in which the task scheduler is simple in terms of logic. Meanwhile, NuttX complies POSIX which makes the scheduler more complex. Secondly, as mentioned before, the switch latency also depends on the port of RTOS on this specific hardware and on the definition of the context for a thread.

The results of thread creation and deletion time costs show that ChibiOS creates and deletes threads faster than NuttX. This is mainly related to the thread structure in the two RTOSs. Threads in NuttX have most common APIs in Linux thus have more attributes than simpler threads in ChibiOS. More memory reading is done which is time-consuming. However, in a well-designed real-time project, threads should not be created or deleted dynamically. Usually, they should be created during the initialization of the system. As we can see, in a contention scenario, mutex acquiring and releasing executions are faster in ChibiOS than those in NuttX. This

Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 09,2022 at 16:31:16 UTC from IEEE Xplore. Restrictions apply.

TABLE II
BEHAVIOURAL TEST RESULTS

Behavioural Tests	NuttX	ChibiOS
Thread creation behaviour		
Lower priority not activated?	YES	YES
Same priority placed at?	HEAD	TAIL
Yielding works?	YES	YES
Higher priority activated?	YES	YES
Round-robin behaviour		
Round-robin scheduling works?	YES	YES
Priority inversion avoidance		
Priority inheritance works?	YES, but with binary semaphore	YES
Semaphore locking and releasing behaviour		
Locking test succeeds?	YES	YES
Releasing test succeeds?	YES	YES

is not surprising because each time cost here includes a thread switch and we already know that the thread switch cost of ChibiOS is lower than NuttX. But the two RTOSs' performances in the non-contention scenario are similar to each other.

We can see in the results that NuttX creates mutexes faster than ChibiOS while ChibiOS performs better at mutex deletion operations. This test results in a tie.

Semaphore acquisition and release performance in the non-contention case of NuttX is better than that of ChibiOS. We can also notice that for both NuttX and ChibiOS, in the non-contention scenario, semaphores acquire and release faster than mutexes. The difference is more remarkable in NuttX. Again, in the contention scenario, because each operation of semaphore acquisition and release includes a thread switch, NuttX is slower than ChibiOS if the semaphore is in contention.

On semaphore creation and deletion operations, NuttX performs better than ChibiOS. But as we mentioned, unlike mutexes, semaphores are usually not created and deleted dynamically while running an application. Thus, the difference in this test between NuttX and ChibiOS is acceptable.

VI. CONCLUSION AND FUTURE WORK

In this paper, we presented a set of tests deliberately designed for scoring real-time operating systems. Specifically, we performed all the tests on NuttX and ChibiOS, two popular RTOSs used on unmanned aerial vehicles and compared the results.

All the tests were done with the system tick interrupt handling on the highest interrupt priority level which is a default setting on most development hardware platforms. This should be considered otherwise by the application designer when constructing proprietary hardware.

Threads with the same priorities given that round-robin is provided, reveals how such a wrong design affects the performance of an RTOS.

The most important aspects of an RTOS are tested, including system tick handling, threads, mutexes and semaphores.

We can state several conclusions from the comparison of the two RTOSs. Clearly, ChibiOS is the winner here since it outperforms NuttX in most temporal tests, even more importantly, succeeds in the priority inversion avoidance test with mutexes rather than binary semaphores. The thread switch overhead of NuttX is significantly larger than it of ChibiOS which augments the probability of a missing deadlines for the same application software. This will more often happen in multithreading designs introducing more context switches. It is also a pity that NuttX does not provide priority inheritance for threads using mutexes. This may cause wrong designs.

NuttX is POSIX compliant, which might be a disadvantage, introducing heavier firmware and larger overhead than ChibiOS which implements its own unique APIs. As drones are usually programmed specifically for dedicated hardware platforms and have strict real-time constraints, a lightweight RTOS with all necessary elements and good performance, like ChibiOS, might be a good choice. The conversion of Ardupilot's from NuttX to ChibiOS is thefore probably a good move.

However, these tests are not telling everything. One of the future works could be the use of a symmetric multiprocessing (SMP) platform and adapted test framework. As the use of multiprocessors becomes more popular even in real-time projects, testing the behaviours and performance of RTOSs on SMP hardware is interesting. For example, for the thread switch test in the SMP scenario, the scheduler must decide on which core should the thread be scheduled. Another possible test could be that threads hopping among cores when there are more running threads than processor cores. Wrong scheduling in this case could lead to missing deadlines. Dedicated Systems Experts provides today these possibilities ready to be used on a SMP platform in their RTOS test framework.

Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 09,2022 at 16:31:16 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PERFORMANCE TEST RESULTS

Performance Tests	NuttX			ChibiOS			#Samples
	Min	Avg	Max	Min	Avg	Max	
System tick overhead (µs)	**3.30**	**3.40**	**3.60**	4.40	4.60	4.70	5000
Thread switch latency							
Average for 2 threads (µs)	16.52	16.53	19.63	**3.57**	**3.60**	**3.62**	5000
Average for 4 threads (µs)	16.36	16.65	19.86	**3.86**	**4.02**	**4.05**	5000
Average for 8 threads (µs)	16.36	18.05	22.49	**3.86**	**4.89**	**4.91**	5000
Average for 16 threads (µs)	16.42	19.24	23.04	**3.86**	**6.61**	**6.65**	4000
Thread creation and deletion time							
Cost of thread creation, never run (µs)	81.62	81.81	86.41	**2.24**	**2.73**	**3.05**	2500
Cost of thread deletion, never run (µs)	19.11	19.72	20.03	**0.83**	**0.83**	**0.84**	2500
Cost of thread creation, run and terminate (µs)	62.06	65.24	70.12	11.12	11.20	11.58	2500
Cost of thread deletion, run and terminate (µs)	**0.51**	**0.59**	**0.76**	0.76	0.80	0.83	2500
Cost of thread creation, run and block (µs)	61.37	61.77	66.62	**11.27**	**11.35**	**11.72**	2500
Cost of thread deletion, run and block (µs)	19.04	19.64	23.98	**0.76**	**0.81**	**0.83**	2500
Mutex acquisition and release time							
Mutex acquisition time: contention (µs)	17.07	17.08	19.90	**5.41**	**5.47**	**5.50**	2500
Mutex release time: contention (µs)	18.93	18.97	22.32	**5.74**	**5.79**	**5.81**	2500
Mutex acquisition time: non-contention (µs)	1.74	1.74	1.96	**1.74**	**1.74**	**1.82**	2500
Mutex release time: non-contention (µs)	1.46	1.48	1.52	**1.36**	**1.37**	**1.43**	2500
Mutex creation and deletion time							
Mutex creation time (µs)	**0.29**	**0.29**	**0.41**	0.48	0.48	0.48	5000
Mutex deletion time, used (µs)	1.05	1.05	1.05	**0.38**	**0.38**	**0.38**	2500
Mutex deletion time, never used (µs)	0.84	0.87	0.93	**0.38**	**0.38**	**0.39**	2500
Semaphore acquisition and release time							
Semaphore acquisition time: contention (µs)	19.55	20.76	25.20	**4.07**	**4.13**	**4.21**	2500
Semaphore release time: contention (µs)	16.19	16.26	19.17	**4.55**	**4.58**	**4.62**	2500
Semaphore acquisition time: non-contention (µs)	**0.37**	**0.37**	**0.57**	1.07	1.08	1.14	2500
Semaphore release time: non-contention (µs)	**0.32**	**0.32**	**0.48**	1.10	1.10	1.12	2500
Semaphore creation and deletion time							
Semaphore creation time (µs)	**0.15**	**0.15**	**0.25**	0.48	0.48	0.52	5000
Semaphore deletion time, used (µs)	**0.20**	**0.20**	**0.30**	0.38	0.38	0.40	2500
Semaphore deletion time, never used (µs)	**0.20**	**0.20**	**0.36**	0.38	0.39	0.50	2500

REFERENCES

[1] B. M. Kumar Reddy, G. S. R. Satyanarayana, and B. Seetaramanjaneyulu, "Scheduling latency comparison of two open-source rtoss on cortex-m3," in *2014 International Conference on Embedded Systems (ICES)*, 2014, pp. 59–62.

[2] J.-W. Lee, S. C. Kim, D.-K. Woo, Y.-S. Ma, and P. Mah, "Performance comparison of mrtos and vxworks in multibench benchmark suite," in *2017 19th International Conference on Advanced Communication Technology (ICACT)*, 2017, pp. 871–873.

[3] (2020) Embedded Microprocessor Benchmark Consortium. [Online]. Available: https://www.eembc.org/

[4] E. Ebeid, M. Skriver, K. H. Terkildsen, K. Jensen, and U. P. Schultz, "A survey of Open-Source UAV flight controllers and flight simulators," *Microprocessors and Microsystems*, vol. 61, pp. 11–20, 2018.

[5] (2019) Apache NuttX. [Online]. Available: http://nuttx.apache.org/

[6] Andrew S. Tanenbaum, *Operating Systems: Design and Implementation, 3rd edition*. Prentice Hall, 2009.

[7] G. D. Sirio. (2006) ChibiOS free embedded RTOS . [Online]. Available: https://www.chibios.org/

[8] Wikipedia, "ChibiOS/RT - Wikipedia," 2020. [Online]. Available: https://en.wikipedia.org/wiki/ChibiOS/RT

[9] H. Fayyad-Kazan, L. Perneel, and M. Timmerman, "Benchmarking the Performance of Microsoft Hyper-V server, VMware ESXi and Xen Hypervisors," *Journal of Emerging Trends in Computing and Information Sciences*, vol. 4, no. 12, pp. 922–933, 2013.

[10] L. Perneel, H. Fayyad-Kazan, L. Peng, F. Guan, and M. Timmerman, "Business Hypervisors for Real-time Applications," *Engineering, Technology & Applied Science Research*, vol. 5, no. 4, pp. 832–840, Aug 2015.

[11] M. T. Hasan Fayyad-Kazan, Luc Perneel, Long Peng, Fei Guan, "RTS hypervisor qualification for real-time systems," *International Journal on Recent and Innovation Trends in Computing and Communication*, vol. 4, no. 8, pp. 147–155, Aug 2016.

[12] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 2018.

[13] G. Reeves and JPL Pathfinder team. (1997) What really happened on Mars? [Online]. Available: https://www.cs.unc.edu/~anderson/teach/comp790/papers/mars_pathfinder_short_version.html

[14] B. W. Lampson and D. D. Redell, "Experience with processes and monitors in Mesa," *Communications of the ACM*, vol. 23, no. 2, pp. 105–117, Feb 1980.

Authorized licensed use limited to: Universita' Politecnica delle Marche. Downloaded on November 09,2022 at 16:31:16 UTC from IEEE Xplore. Restrictions apply.